

Static analysis with focus on using language models and Hoare logic

Rashid Harvey
Freie Universität Berlin
IoT & Security Seminar Report

Abstract—IoT devices are deployed at enormous scale, are often too resource-constrained to defend at runtime, and are rarely patched, which makes establishing software correctness *before* deployment especially valuable. This report surveys three families of technique for doing so: lightweight static analysis (lexical and type-based checks, information-flow analysis, and memory safety by construction in Rust), formal methods (Hoare logic and verification-aware tools such as Dafny and hax), and the emerging use of large language models to drive formal verification. The headline numbers reported on benchmarks differ sharply from behavior on real, security-critical code, both in industrial static-analysis deployments and in recent AI-assisted verification results. We argue that the foundational ideas behind these techniques are largely decades old, and that what is now changing is their *ease of use*. Pairing the productivity of language models with the soundness of a deterministic verifier could lower the usability barrier that has kept formal guarantees out of everyday practice, though current evidence shows it works far better on small benchmarks than on large systems.

Index Terms—static analysis, formal verification, Hoare logic, large language models, IoT security

I. INTRODUCTION

A. IoT Security

The Internet of Things (IoT) has grown into one of the largest computing domains in existence. Industry trackers estimate that the number of connected IoT devices reached roughly 21 billion worldwide in 2025, growing at about 14% per year and projected to approach 39 billion by 2030, [1] as the worldwide IoT market continues its rapid expansion. [2] This ubiquity has not gone unnoticed by attackers: already in 2016, Gartner predicted that “by 2020, more than 25 percent of identified attacks in enterprises will involve IoT, although IoT will account for less than 10 percent of IT security budgets.” [3]

Indeed, security is frequently an afterthought in IoT development. Profit-driven business models and pressure to minimize time-to-market lead manufacturers to overlook security considerations and to ship devices that are vulnerable by design. [4]

Compounding this, IoT devices commonly run custom-built, vendor-specific firmware rather than the standardized, widely-audited software stacks found in general-purpose computing. The resulting heterogeneity of standards and communication stacks means that traditional security countermeasures cannot be directly applied to IoT technologies, [5] and the inherent complexity of so many heterogeneous entities interacting

greatly complicates the design and deployment of interoperable security mechanisms. [6] Bespoke firmware is also more prone to weak programming practices and unaudited code paths than mature, mainstream software. [4]

A further obstacle is that IoT endpoints are typically severely resource-constrained. The IETF characterizes such “constrained nodes” by hard limits on memory (RAM), code size (ROM/Flash), available computation, and power, with the smallest device class offering well under 10 KiB of RAM. [7] Generally, “limited storage, power, and computational capabilities hinder addressing IoT security requirements.” [4]

Finally, even once vulnerabilities are discovered, IoT devices are notoriously difficult to update. Most devices lack a built-in secure firmware update mechanism, so that “critical security vulnerabilities cannot be fixed, and the IoT devices can become a permanent liability.” [8] Many are deployed with little or no infrastructure for applying patches, [9] and even where update mechanisms exist they are routinely neglected: a large share of devices run outdated firmware, with surveys reporting that roughly 40% of users never update at all. [4]

B. Static Analysis

Static analysis is the analysis of code, binaries or other artifacts to check certain properties without, in contrast to dynamic analysis, executing any code. [10]

Static analysis examines code *without* executing it; dynamic analysis examines it *by* executing it.

- **Examples for static analysis:** linting, formatting, syntax checking, type checking, flow analysis.
- **Examples for dynamic analysis:** testing, emulation.

Static analysis is efficient, i.e. it runs quickly and cheaply, and catches errors early, before runtime. Its limitations are of a theoretical nature (Rice’s theorem, of which the halting problem is the classic special case) and a potentially high false-positive rate.

1) **Case studies:** The practical value of static analysis is best seen at industrial scale, where three widely-cited reports converge on the same lesson: the hard part is not finding bugs but getting developers to act on them, which makes the false-positive rate and workflow integration decisive.

Commercializing the Coverity analyzer across billions of lines of real-world C/C++, Bessey et al. found that developers quickly abandon a checker that reports too many spurious warnings, so a low false-positive rate matters more than raw analytical power [11].

Google reached the same conclusion with its Tricorder platform: analyses are surfaced inside the code-review tool, held to an effective false-positive budget of around 10%, and let reviewers flag unhelpful results so that noisy checks are disabled [12].

Facebook’s Infer, whose roots lie in separation-logic research, is instead deployed at “diff time”: it runs automatically when a change is submitted for review, so issues reach developers promptly, which the authors report is key to high fix rates [13]. Across all three, effective static analysis turns out to be as much a matter of engineering and developer experience as of program-analysis theory.

C. Large Language Models for Coding

Large language models (LLMs) have seen massive advancements for coding and security. When the SWE-bench benchmark of real GitHub issues was introduced in 2023, the best-performing model, Claude 2, resolved only 1.96% of its 2,294 tasks. [14] Only a few years later the picture is transformed: on the smaller, human-curated SWE-bench Verified subset (a different test set, so the two figures are not directly comparable), Claude Fable 5 reports a 95% resolution rate at the model’s release. [15]

Even accounting for the limited reliability of public benchmarks,¹ this still indicates that LLMs have become genuinely useful for writing code. Producing code that passes tests is, however, a weaker requirement than producing a machine-checked proof; as the verification case studies of Section IV show, generating proof artifacts is markedly harder.

These models are now capable enough to draw a regulatory response. On the 12th of June 2026, the US government forced Anthropic to withdraw Fable 5, the more broadly accessible counterpart to the access-restricted Mythos 5, from public access after about 76 hours, by imposing a national-security export control. [18]

This capability extends beyond writing new code: the security firm AISLE reports having used a Claude-based system to uncover several previously-unknown vulnerabilities, including a high-severity CVE, in famously hard-audited, mature code-bases such as OpenSSL. [19]

II. STATIC ANALYSIS METHODS

A. Lexical Methods and Methods Using the Abstract Syntax Tree Representation

Lexical methods and methods using the abstract syntax tree (AST) representation are pretty “simple” methods, which you would often summarize as linting.

Lexical analysis simply means breaking down text into the smallest semantic units, “tokens”, and analyzing them. The AST is a representation of the code as a tree, which allows for more depth (see Fig. 1). The AST is what a compiler, transpiler, interpreter and most linters and formatters will consume (see Fig. 2).

¹Benchmarks can leak training data: filtering suspected leakage dropped one agent’s score from 12.47% to 3.97% [16], and models may memorize solutions rather than learn the underlying principles [17].

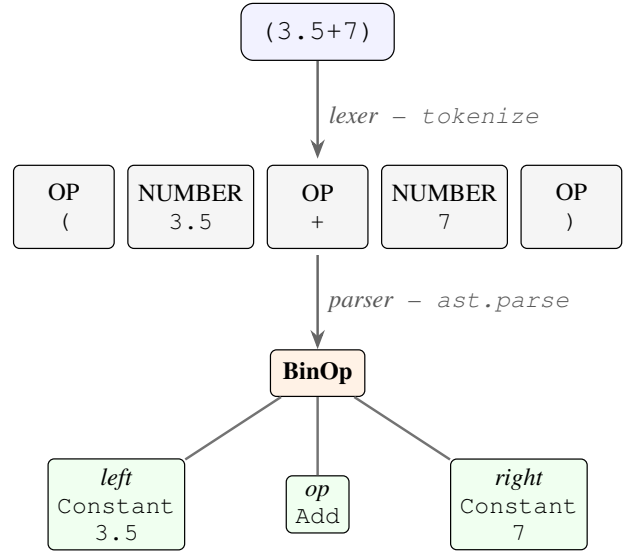


Fig. 1. Lexing and parsing of the expression $(3.5+7)$ in Python

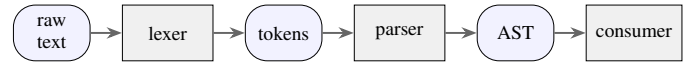


Fig. 2. Front-end pipeline of a static analyzer. [20] Rounded nodes are data, rectangular nodes are processing stages.

Examples

- 1) Syntax errors
- 2) Name errors or misspellings
- 3) Missing definitions and declarations
- 4) Rule matching, e.g. leaked secrets

B. Type checking

The idea behind types is that any data on a digital computer is simply some bitstring. The interpretation of this bitstring is what makes the data a value.

The data only makes sense if we define operations on it, but these operations depend on the type. For example, in Python the $+$ operator for integers does integer addition, but for strings it does string concatenation. However, if you try to “add” an integer with a string, you get a type error. In Python type checking is done at runtime and is therefore called “dynamic type checking”. However, for static analysis, we can check the types ahead of run-time, “static type checking”. [21], [22]

In his paper “A Theory of Type Polymorphism in Programming” Robin Milner argues that well-typed programs cannot “go wrong” and that therefore programmers should use a formal type discipline. [23]

C. Flow Control

A further class of semantic methods reasons about how information moves through a program. *Information-flow analysis* tracks how the values of variables depend on one another, with the goal of certifying that secret data never reaches a public output. [24], [25] A flow from a variable x to a variable y ,

written $x \rightarrow y$, means that observing y may reveal something about x . Two kinds of such flows are distinguished. [26]

1) **Explicit flows**: The most direct case is an assignment that copies a value. In the program $y := x$ all information from x flows to y . The flow need not be total: in $y := \lfloor x/8 \rfloor$ not all, but some information may flow from x to y , since y no longer determines x exactly. Both are *explicit* flows, denoted $x \rightarrow y$, because the dependency is carried directly by the data passed in the assignment.

2) **Implicit flows**: Information can also leak through the control flow itself, without any direct assignment from the source. Consider:

```
y := 0
if x ≥ 8 then
  y := 1
end if
```

Here x is never assigned to y , yet some information still flows from x to y : after the final line, observing y tells us whether $x \geq 8$. This is an *implicit* flow, again denoted $x \rightarrow y$. In general, any conditional control-flow change encodes information about the values that guided it, so a sound analysis must account for implicit as well as explicit flows. [24], [25]

3) **Practical enforcement tools**: build on this distinction; the Jif/JFlow extension of Java, for example, lets programmers annotate data with security labels that the compiler then checks against both flow kinds. [27]

4) **Related flow-analysis techniques**: include control-flow analysis, which examines the possible paths of execution, and taint analysis, which follows untrusted (“tainted”) inputs to ensure they cannot reach sensitive operations unsanitized.

D. Case study: Rust

Rust is a low-level programming language started in 2006 by Graydon Hoare as a side project, then officially sponsored by Mozilla in 2009 and it finally reached maturity (v1.0) in 2015. Rust was specifically created to solve the flaws of C/C++. [28], [29] This section tries to summarize how this is done in Rust.

“Who doesn’t want memory safety, and fast performance, and a friendly compiler, and great tooling, among a host of other wonderful features?” [30]

The Rust language has a very strict compiler which includes not just type checking but also a borrow checker and more. [30] These checks lead to the conclusion that regular Rust code compiled through this strict compiler is also “safe”: “If it compiles, it works” is common community lore. This recalls Milner’s idea that well-typed programs cannot go wrong. [23]

“Rust’s type system and runtime guarantee the absence of data races, buffer overflows, stack overflows, and accesses to uninitialized or deallocated memory.” [31]

The safety and soundness of a core subset of Rust was machine-checked in 2018 in the paper “RustBelt: Securing the Foundations of the Rust Programming Language”, a formal soundness proof that also covers the encapsulation of well-typed *unsafe* code. [32] These guarantees apply to *safe* Rust;

the *unsafe* escape hatch must be checked by hand, which is exactly what such proofs formalize.

1) **Case studies**: Large-scale industry data supports this: Microsoft reports that roughly 70% of its annually assigned CVEs are memory-safety issues, [33], [34], while Android saw memory-safety vulnerabilities fall from 76% to 24% after shifting new development to memory-safe languages. [35]

III. FORMAL METHODS

“Classical static analysis finds symptoms; formal methods prove properties.” Provable security is a desirable goal: it would, for example, remove the need to ship security patches at all. It also has real limitations, covered below.

A. Formal Models and Formal Verification

When applying formal methods, a formal model is created first; the kind of model depends on the use case. The desired properties are then checked against that model, for example by model checking, deductive verification (such as Hoare logic), or interactive theorem proving. This checking step is what is called formal verification.

B. Hoare Logic

Hoare logic is a program logic in which a fragment of code C is annotated with a *precondition* P and a *postcondition* Q , written as the Hoare triple

$$\{P\} C \{Q\}.$$

Its meaning is that if P holds before C executes, then Q is guaranteed to hold afterwards. For example,

$$\{x < 5\} \ x \ += \ 1 \ \{x < 6\}.$$

1) **Loops**: Loops are reasoned about through a *loop invariant*: a property that holds before and after every iteration. For a *while* loop, the postcondition is the invariant conjoined with the negated loop condition, which necessarily fails once the loop terminates. A *for* loop can be *unrolled* when its iteration count is statically known, or treated as an iteration over a data structure. For example Gaussian sum over y or mapping $*2$ over a *list*:

```
{x = 0 ∧ y > 0}
for i in range(y+1): x += i
{x = y(y+1)/2 ∧ y > 0}

{∀i. x[i] ≥ 0}
for i in range(len(x)): x[i] *= 2
{∀i. x[i] ≥ 0}
```

A *while* example computing the Gaussian sum makes the role of the invariant explicit:

```
{y ≥ 0}
x = 0; i = 0
inv {x = i(i+1)/2 ∧ i ≤ y}
while i < y: i += 1; x += i
{x = y(y+1)/2}
```

```

method ISqrt(n: int) returns (r: int)
  requires 0 ≤ n;
  ensures r*r ≤ n ∧ n < (r+1)*(r+1);
  { /* method body goes here... */ }

```

Fig. 3. A Dafny method specification: `ISqrt` returns the integer square root r of a non-negative input n [45].

The invariant holds on entry (where $x = i = 0$) and is preserved by every iteration; on exit the loop condition fails, so $i = y$, and the invariant conjoined with this negated condition collapses to the postcondition $x = y(y + 1)/2$.

2) **Usage:** Hoare logic can be used to prove the correctness of algorithms which M. Foley and C.A.R. Hoare demonstrated quite nicely in “Proof of a Recursive Program: Quicksort” (1971). [36]

C. Overview of Methods and Tools

- 1) Model-based formal specification languages [37] like Z [38], the Vienna Development Method (VDM) [39], and B [40]
- 2) Interactive proof assistants [41] like Rocq (formerly Coq), [42] Isabelle/HOL [43] or Lean [44]
- 3) Verification-aware programming languages [41] like Dafny [45], SPARK [46], F* [47], Why3 [48], Viper [49], Why3 [50] or hax [51]
- 4) Crypto verifiers like ProVerif [52] and EasyCrypt [53]

D. Examples

1) **Z:** The Z notation (pronounced “zed”) is a formal specification language for describing and modelling computing systems [38], [54]. It uses mathematics, mainly sets and logic, to write a precise, unambiguous description of what a system should do, before any code is written. Its main building block is the *schema*: a named box that groups some variables together with rules about how they may behave, and schemas can be combined to build up larger specifications. Z is not executable; it describes a system rather than implementing it. It has been applied to real industrial systems, most famously the formal specification of IBM’s CICS transaction-processing software, work that earned Oxford University and IBM a Queen’s Award for Technological Achievement in 1992.

2) **Dafny:** In the original paper on Dafny, Leino gave the challenges of directly using interactive proof assistants and satisfiability-modulo-theories (SMT) solvers as main reason for the creation of Dafny. The idea is that the programmer doesn’t have to interact with any proof assistant or solver directly, only needs to make certain annotations. The original paper shows the simple usage: You define a `method` and pre- and postconditions (see Fig. 3), just like in Hoare logic. [45].

Dafny also compiles down to C#, Java, JavaScript, Go, and Python. A more complete algorithm for creating the Dutch flag can be found in Fig. 4. [55].

3) **hax:** Hax is a tool for proving code written in Rust. It translates a large (though not complete) subset of Rust into formal languages such as F* or Rocq, where security

```

1 method Main()
2 {
3   var a := new Color[3];
4   a[0] := White;
5   a[1] := Blue;
6   a[2] := Red;
7   assert a[..] == [White, Blue, Red];
8   DutchFlag(a);
9   assert Ordered(a[0], a[1]);
10  assert Ordered(a[1], a[2]);
11  assert a[..] == [Red, White, Blue];
12 }
13
14 datatype Color = Red | White | Blue
15
16 predicate Ordered(c: Color, d: Color)
17 {
18   c == Red ||
19   c == d ||
20   d == Blue
21 }
22
23 method DutchFlag(a: array<Color>)
24   requires a != null modifies a
25   ensures forall i, j :: 0 <= i < j < a.Length ==> Ordered(a[i], a[j])
26   ensures multiset(a[..]) == old(multiset(a[..]))
27 {
28   var r, w, b := 0, 0, a.Length;
29   while w != b
30     invariant 0 <= r <= w <= b <= a.Length;
31     invariant forall i :: 0 <= i < r ==> a[i] == Red
32     invariant multiset(a[..]) == old(multiset(a[..]))
33     { match a[w]
34       case Red =>
35         a[r], a[w] := a[w], a[r];
36         r, w := r + 1, w + 1;
37       case White =>
38         w := w + 1;
39       case Blue =>
40         b := b - 1;
41         a[w], a[b] := a[b], a[w];
42     }
43 }

```

Fig. 4. A fully verified Dafny program (the Dutch national flag); the IDE gutter shows the per-line verification status [55].

properties and functional correctness can then be discharged with the appropriate prover [51], [56].

Hax is a verification toolchain aimed at security-critical software such as cryptographic libraries, protocol implementations, and parsing or sanitization code. Its guiding observation is that different verification tools excel at different goals, so rather than committing to a single prover it extracts the supported subset of a Rust program and translates it into the input language of whichever backend fits the property at hand: domain-specific security analyzers like ProVerif for protocol- and cryptography-level security arguments, and general proof assistants like F* and Rocq for functional correctness. Because soundness then rests on these translations, the authors systematically test both the generated models and their models of the Rust system libraries against the original code. [51]

A flagship application of this toolchain is Bertie (“Bert13”), a portable post-quantum TLS 1.3 implementation in Rust that hax verifies using F*, ProVerif, and SSProve for runtime safety, parsing correctness, and protocol security. [57]

4) **ProVerif:** ProVerif is a tool for the automatic, symbolic verification of cryptographic protocols [52], [58]. It works in the Dolev-Yao model, in which cryptography is assumed to be perfect and treated as a black box, while the attacker is given full control of the network: messages can be intercepted,

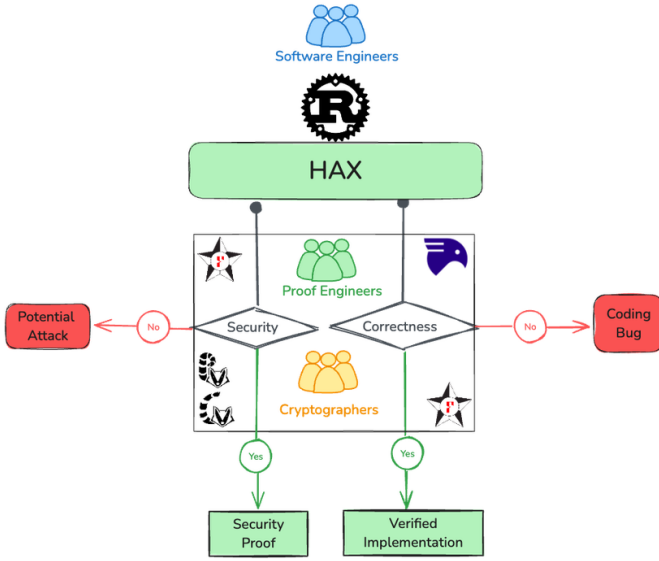


Fig. 5. The hax workflow: Rust code is routed through hax to separate security and correctness proofs, each verified via prover backends [56].

modified, replayed, or injected, but the encryption itself cannot be broken. Given a description of a protocol, ProVerif checks security properties such as secrecy (whether an attacker can learn a secret value) and authentication. It has been used to verify and improve real-world protocols, including Signal, TLS, avionic protocols, and internet voting. Unlike Dafny or hax, which reason about the correctness of code, ProVerif reasons about the security of a protocol’s design, which is why hax can target it as one of its backends [51].

E. Limitations

Despite these successes, a recent comprehensive review of formal methods for IoT identifies several recurring gaps that still limit their adoption [37]:

- **Scalability.** As IoT networks grow, so does the state space a verifier must explore; techniques such as model checking quickly run into state-space explosion and become computationally infeasible for large-scale deployments.
- **Real-time performance.** Formal methods do not yet fully capture the timing behavior of IoT systems, where processing delays and network latency threaten strict deadlines, a serious obstacle in time-sensitive domains such as healthcare and industrial automation.
- **Cooperation between methods.** Model checking, theorem proving, and formal specification have complementary strengths, but there is little work on integrating them; the absence of frameworks that combine these techniques prevents holistic verification of both functional and non-functional requirements.
- **Adaptability.** IoT systems evolve constantly as devices are added, updated, or removed, whereas most formal models are static and cannot track such changes without full re-verification.

- **Security and privacy.** Much of the literature concentrates on functional correctness, leaving security- and privacy-specific properties comparatively under-addressed despite the sensitivity of IoT data.
- **Usability.** The mathematical intricacy of formal methods keeps them out of reach for most practitioners; more accessible, developer-friendly tooling is needed before they see widespread industrial use.

IV. ARTIFICIAL INTELLIGENCE FOR FORMAL VERIFICATION

The usability and scalability gaps just described have made formal verification a natural target for machine learning. Deep-learning methods, and more recently LLMs, are now applied across the proving workflow, from autoformalization and premise selection to generating individual proof steps and searching for whole proofs [59]. The appeal follows directly from the barrier identified above: if a model can supply the tedious invariants, annotations, and proof scripts that formal tools demand, verification becomes accessible to developers who are not themselves proof experts. A prominent demonstration is AlphaProof, an AlphaZero-style agent that learns to write formal Lean proofs through reinforcement learning and, together with AlphaGeometry 2, reached silver-medal-level performance at the 2024 International Mathematical Olympiad [60].

The pairing of AI with formal methods is complementary in both directions. LLMs are powerful but non-deterministic: they produce plausible proofs or code with no guarantee of correctness. A formal verifier supplies what they lack, a deterministic oracle that either accepts a proof or rejects it with a concrete error. It grounds the model’s output in a checkable truth and closes a tight feedback loop, since the prover’s error can be fed straight back as the next prompt until the proof checks. Because soundness rests on the verifier rather than the model, an unreliable generator can still yield a trustworthy result, but only relative to the specification it is checked against. This caveat matters: a proof of a wrong or vacuous specification is worthless, and models are known to “game” the checker by weakening postconditions or inserting unproven assumptions, a failure mode that recent evaluations document explicitly [61]. Writing a correct, complete specification is itself hard, and where the model is asked to supply the specification as well as the proof [62] the verifier only checks the code against the model’s own claim, so the guarantee becomes circular. The residual difficulty therefore moves to the specification: a developer who is not a verification expert may be unable to distinguish a faithful specification from a weakened one, so for that audience these tools shift the effort from writing proofs to reviewing specifications rather than removing it. In the other direction, formal modelling is notoriously tedious; hand-writing specifications, invariants, and proof scripts is much of why these techniques, despite their guarantees, are so seldom used in practice. That is the kind of laborious, pattern-heavy work LLMs handle well, so a model can take on the bulk of

it and leave the engineer to supervise, easing the usability gap identified earlier.

A. Case studies

Most current work attaches an LLM to an existing verifier and asks it to produce the artifacts that verifier needs. For Rust checked with the Verus tool, Yao et al. combined GPT-4 with static analysis to synthesize the loop invariants and assertions Verus requires, easing the proof burden on small programs [63]. AutoVerus scales this idea up with a network of orchestrated LLM agents that imitate an expert’s generate, refine, and debug loop, solving more than 90% of a 150-task benchmark [64].

Dafny has received comparable attention. Misu et al. ran the first empirical study of prompting LLMs to synthesize Dafny methods *together with* their formal specifications [62], while Laurel uses domain-specific prompting to generate the helper assertions Dafny’s SMT backend needs, recovering roughly 56% of the assertions in a real-world dataset [65].

In interactive proof assistants the unit of automation is larger. Baldur fine-tunes an LLM to generate entire Isabelle/HOL proofs in a single shot, and then to repair them from the prover’s own error output, outperforming search-based methods [66]. PALM takes a generate-then-repair approach for Coq, pairing the model with symbolic tactics to close proofs it cannot produce directly [67].

Closest to the security-critical setting of this report, a recent experience report revisits the hax toolchain seen earlier but replaces its classical backends with AI provers. It translates production Rust cryptographic code (zkEVM primitives in Plonky3 and RISC Zero) into Lean 4, recording which proof obligations the AI could discharge and where human effort was still required [68]. Where Bertie verifies a complete TLS stack with classical provers, this pipeline probes how far AI provers can carry the same toolchain on real code.

These results are encouraging, but they are largely measured on benchmarks. DafnyBench, a suite of roughly 750 programs, reports a best success rate near 68% for LLM-generated verification hints [69]. On the genuinely large, Coq-verified FSCQ file system, however, the same class of models reproduces only about 38% of the proofs, with scale and lemma retrieval becoming the bottleneck [70]; a broader case study across proof assistants reaches the same conclusion, cataloguing the many distinct failure modes current models exhibit [61]. This mirrors the industrial static-analysis experience seen earlier: a benchmark score and the behavior on real, security-critical code can differ sharply, and for IoT it is the real-code behavior that matters.

V. DISCUSSION

Formal verification tools and, more generally, static analysis tools are mostly “solved”, in the sense that the ideas still in practice today are from the late 1960s to the 1980s. [24], [71], [72] The things that change are the applications, programming languages and the ease-of-use.

For formal verification to become more widely adopted, not only must ease-of-use improve, but legal requirements, standards, and consequences must increase as well. Recent measures push in this direction: the EU Cyber Resilience Act, which mandates security-by-design and vulnerability handling for connected products and backs them with substantial fines, and consumer-IoT baselines like ETSI EN 303 645. [73], [74] Until such requirements and their consequences are stronger, the incentives are simply not enough.

However, LLMs and new more advanced auto-solvers like hax [51] or Dafny [45] provide the possibilities to prove more-and-more and larger-and-larger codebases. One limitation that is still open however is the **cooperation between methods** [37] and the **hardware interoperability**.

For cooperation, tools like hax and Dafny that think this abstraction from the beginning, are very valuable: hax compiles Rust to several verification systems [56] and Dafny compiles to several different programming languages [55]. They are therefore already highly adaptable.

To adapt better to hardware specifics, one direction is to specify the hardware itself in a formal language rather than emulating it with tools like QEMU, a kind of *theoretical emulator*, so that the same model serves both execution and proof. This is not hypothetical: architecture-definition languages such as Sail already generate both executable emulators and proof-assistant models of an instruction set (e.g. ARM and RISC-V) from a single formal specification [75]; the open problem is extending this style of hardware/software co-verification to the heterogeneous, constrained processors typical of IoT.

VI. CONCLUSION

This report followed a single question across three families of technique: how can we gain confidence that software is correct and secure *before* it runs? The question is highly relevant to IoT, whose tens of billions of devices are often too numerous to supervise, too constrained to defend at runtime, and too rarely updated to fix after the fact.

Static analysis sits at one end of the spectrum. It is relatively cheap and fast, and it can flag whole classes of defect, from syntax and type errors to information-flow leaks, without ever executing a line. Its reach has grown from lexical checks over token streams to semantic reasoning about types and data flow, and languages such as Rust now push much of it into the compiler itself, making memory-safety errors unrepresentable by construction. Yet the main lesson from deploying these tools at industrial scale is not about analytical power at all: Coverity, Google, and Facebook each found that adoption hinges on low false-positive rates and tight integration into the developer’s workflow.

Formal methods sit at the other end; they *prove* the absence of whole classes of fault instead of just flagging symptoms. Hoare logic shows how a precondition, a program, and a postcondition can be tied into a checkable guarantee, and modern verification-aware languages turn that idea into practical tooling: Dafny discharges proof obligations through an SMT backend, while hax carries security-critical Rust into provers

such as F* and Rocq, the foundation on which the Bertie post-quantum TLS stack was fully verified with no language models in the loop. The catch, once again, is adoption. A recent review of formal methods for IoT catalogues six recurring gaps: scalability, real-time behavior, cooperation between methods, adaptability, a thin focus on security and privacy, and, above all, usability. It is the sheer tedium of hand-writing specifications and proofs that keeps these guarantees out of everyday practice.

Language models are now being applied to exactly this gap. In only a few years LLMs have gone from solving almost none of the SWE-bench issues to nearly all of them, and a related, more demanding, generative ability is now being turned on verification itself: tools such as AutoVerus, Laurel, Baldur, and PALM let a model supply the invariants, assertions, and proof scripts that formal engines demand. The pairing helps both sides: the model gains a deterministic oracle to check against, and in return it absorbs the tedious annotation work that made formal methods unpalatable. Soundness still rests on the verifier rather than the model, so even an unreliable generator can yield a trustworthy result, as long as the specification it is checked against is correct. The honest caveat is that today's strong numbers are mostly benchmark numbers: on genuinely large, security-critical code such as the FSCQ file system the success rate falls sharply, the benchmark-versus-reality gap that the static-analysis case studies warned about.

A promising path for IoT is to combine techniques rather than rely on any one: the productivity of AI together with the soundness of a formal engine, which would lower the usability barrier that has long kept verification away from constrained, unpatchable devices. Two problems remain: First, the evidence is not sufficient: results are strong on small benchmarks but fall off sharply on large, real systems, and none of the AI-assisted verification surveyed here has yet been demonstrated on IoT firmware, so this is a direction rather than a settled result. Second, usability is necessary but not sufficient for adoption: as the introduction argued, IoT insecurity is driven as much by economic incentives and an unpatchable installed base as by the cost of proof, and development-time verification helps only the devices still to be built. With those caveats, the tools best placed to exploit this composition are those built for cooperation from the start, such as hax compiling Rust to several provers and Dafny compiling to several languages. The remaining frontier is the boundary with hardware, where formal models of an instruction set in the spirit of Sail could one day let software and hardware be verified against a single specification. Almost none of the underlying theory is new; the lexical, type-theoretic, flow-based, and Hoare-style ideas surveyed here date from the late 1960s to the 1980s. What is finally changing is their ease of use. On its own, cheaper proof will not move an industry whose incentives point elsewhere; but combined with the legal and economic pressure discussed above, lowering the cost of correctness is a necessary part of any durable solution for IoT devices that cannot be patched into safety.

REFERENCES

- [1] S. Sinha, “State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally,” <https://iot-analytics.com/number-connected-iot-devices/>, Oct. 2025.
- [2] Statista, “Internet of Things — worldwide (market outlook),” <https://www.statista.com/outlook/tmo/internet-of-things/worldwide>, 2025.
- [3] Gartner, “Gartner says worldwide IoT security spending to reach \$348 million in 2016,” <https://www.gartner.com/en/newsroom/press-releases/2016-04-25-gartner-says-worldwide-iot-security-spending-to-reach-348-million-in-2016>, Apr. 2016.
- [4] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, and N. Ghani, “Demystifying IoT security: An exhaustive survey on IoT vulnerabilities and a first empirical look on internet-scale IoT exploitations,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2702–2733, 2019.
- [5] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, “Security, privacy and trust in Internet of Things: The road ahead,” *Computer Networks*, vol. 76, pp. 146–164, 2015.
- [6] R. Roman, J. Zhou, and J. Lopez, “On the features and challenges of security and privacy in distributed Internet of Things,” *Computer Networks*, vol. 57, no. 10, pp. 2266–2279, 2013.
- [7] C. Bormann, M. Ersue, and A. Keränen, “Terminology for constrained-node networks,” RFC 7228, Internet Engineering Task Force (IETF), May 2014.
- [8] K. Zandberg, K. Schleiser, F. Acosta, H. Tschofenig, and E. Baccelli, “Secure firmware updates for constrained IoT devices using open standards: A reality check,” *IEEE Access*, vol. 7, pp. 71 907–71 920, 2019.
- [9] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the Mirai botnet,” in *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, 2017, pp. 1093–1110.
- [10] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Berlin, Heidelberg: Springer, 1999.
- [11] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, “A few billion lines of code later: Using static analysis to find bugs in the real world,” *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.
- [12] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, “Lessons from building static analysis tools at google,” *Communications of the ACM*, vol. 61, no. 4, pp. 58–66, 2018.
- [13] D. Distefano, M. Fähndrich, F. Logozzo, and P. W. O’Hearn, “Scaling static analyses at facebook,” *Communications of the ACM*, vol. 62, no. 8, pp. 62–70, 2019.
- [14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [15] Anthropic, “Claude fable 5 and claude mythos 5 system card,” <https://www-cdn.anthropic.com/d00db56fa754a1b115b6dd7cb2e3c342ee809620.pdf>, 2026.
- [16] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang, “SWE-Bench+: Enhanced coding benchmark for LLMs,” *arXiv preprint arXiv:2410.06992*, 2024.
- [17] S. Liang, S. Garg, and R. Zilouchian Moghaddam, “The SWE-bench illusion: When state-of-the-art LLMs remember instead of reason,” *arXiv preprint arXiv:2506.12286*, 2025.
- [18] Anthropic, “Statement on the US government directive to suspend access to fable 5 and mythos 5,” <https://www.anthropic.com/news/fable-mythos-access>, Jun. 2026.
- [19] AISLE, “AISLE discovered 12 out of 12 OpenSSL vulnerabilities,” <https://aisle.com/blog/aisle-discovered-12-out-of-12-openssl-vulnerabilities>, Jan. 2026.
- [20] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Pearson/Addison-Wesley, 2006.
- [21] B. C. Pierce, *Types and Programming Languages*. Cambridge, MA: MIT Press, 2002.
- [22] Python Software Foundation, “The python language reference: Data model,” <https://docs.python.org/3/reference/datamodel.html>, 2026.

- [23] R. Milner, “A theory of type polymorphism in programming,” *Journal of Computer and System Sciences*, vol. 17, no. 3, pp. 348–375, 1978.
- [24] D. E. Denning and P. J. Denning, “Certification of programs for secure information flow,” *Communications of the ACM*, vol. 20, no. 7, pp. 504–513, 1977.
- [25] A. Sabelfeld and A. C. Myers, “Language-based information-flow security,” *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.
- [26] V. Roth, “Information flow control,” Lecture 10, “Information Security” course, Freie Universität Berlin, 2025.
- [27] A. C. Myers, “JFlow: Practical mostly-static information flow control,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’99)*. ACM, 1999, pp. 228–241.
- [28] C. Thompson, “How Rust went from a side project to the world’s most-loved programming language,” <https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/>, Feb. 2023.
- [29] G. Hoare, “10 years of stable rust: An infrastructure story,” <https://rustfoundation.org/media/10-years-of-stable-rust-an-infrastructure-story/>, May 2025.
- [30] S. Klabnik and C. Nichols, *The Rust Programming Language*, 3rd ed. No Starch Press, 2025.
- [31] N. D. Matsakis and F. S. Klock II, “The Rust language,” in *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT ’14)*, ser. Ada Letters, vol. 34, no. 3. ACM, 2014, pp. 103–104.
- [32] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the foundations of the Rust programming language,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 1–34, 2018.
- [33] M. Miller, “Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape,” Presentation at BlueHat IL 2019; slides at <https://github.com/microsoft/MSRC-Security-Research>, Microsoft Security Response Center, Feb. 2019.
- [34] Microsoft Security Response Center, “We need a safer systems programming language,” <https://msrc.microsoft.com/blog/2019/07/we-need-a-safer-systems-programming-language/>, Jul. 2019.
- [35] J. Vander Stoep and A. Rebert, “Eliminating memory safety vulnerabilities at the source,” <https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html>, Sep. 2024.
- [36] M. Foley and C. A. R. Hoare, “Proof of a recursive program: Quicksort,” *The Computer Journal*, vol. 14, no. 4, pp. 391–395, 1971.
- [37] I. Haddou-Oumouloud, A. Kriouile, S. Hamida, and A. Ettalbi, “Toward secure and reliable IoT systems: A comprehensive review of formal methods applications,” *IEEE Access*, vol. 12, pp. 171 853–171 875, 2024.
- [38] J. M. Spivey, *The Z Notation: A Reference Manual*, 2nd ed. Prentice Hall, 1992.
- [39] C. B. Jones, *Systematic Software Development Using VDM*, 2nd ed. Prentice Hall, 1990.
- [40] J.-R. Abrial, *The B-Book: Assigning Programs to Meanings*. Cambridge University Press, 1996.
- [41] S. Zetsche and Y. Moy, “Dafny 2026: Auto-active program verification (workshop co-located with popl 2026),” <https://popl26.sigplan.org/home/dafny-2026>, 2026.
- [42] The Rocq Development Team, “The rocq prover (formerly coq), version 9.0,” <https://rocq-prover.org/>, 2025.
- [43] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, ser. Lecture Notes in Computer Science. Springer, 2002, vol. 2283.
- [44] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, “The Lean theorem prover (system description),” in *Automated Deduction — CADE-25*, ser. Lecture Notes in Computer Science, vol. 9195. Springer, 2015, pp. 378–388.
- [45] K. R. M. Leino, “Dafny: An automatic program verifier for functional correctness,” in *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, ser. Lecture Notes in Computer Science, E. M. Clarke and A. Voronkov, Eds., vol. 6355. Springer, 2010, pp. 348–370.
- [46] J. W. McCormick and P. C. Chapin, *Building High Integrity Applications with SPARK*. Cambridge University Press, 2015.
- [47] N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin, “Dependent types and monadic effects in F*,” *ACM SIGPLAN Notices (POPL 2016)*, vol. 51, no. 1, pp. 256–270, 2016.
- [48] J.-C. Filliâtre and A. Paskevich, “Why3 — where programs meet provers,” in *Programming Languages and Systems (ESOP 2013)*, ser. Lecture Notes in Computer Science, vol. 7792. Springer, 2013, pp. 125–128.
- [49] P. Müller, M. Schwerhoff, and A. J. Summers, “Viper: A verification infrastructure for permission-based reasoning,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2016)*, ser. Lecture Notes in Computer Science, vol. 9583. Springer, 2016, pp. 41–62.
- [50] D. J. Pearce and L. Groves, “Whiley: A platform for research in software verification,” in *Software Language Engineering (SLE 2013)*, ser. Lecture Notes in Computer Science, vol. 8225. Springer, 2013.
- [51] K. Bhargavan, M. Buyse, L. Franceschino, L. L. Hansen, F. Kiefer, J. Schneider-Bensch, and B. Spitters, “hax: Verifying security-critical Rust software using multiple provers,” in *Verified Software. Theories, Tools and Experiments (VSTTE 2024)*, ser. Lecture Notes in Computer Science, vol. 15525. Springer, 2025, pp. 96–119.
- [52] B. Blanchet, “An efficient cryptographic protocol verifier based on Prolog rules,” in *Proceedings of the 14th IEEE Computer Security Foundations Workshop (CSFW-14)*. IEEE, 2001, pp. 82–96.
- [53] G. Barthe, B. Grégoire, S. Heraud, and S. Zanella Béguelin, “Computer-aided security proofs for the working cryptographer,” in *Advances in Cryptology — CRYPTO 2011*, ser. Lecture Notes in Computer Science, vol. 6841. Springer, 2011, pp. 71–90.
- [54] Wikipedia contributors, “Z notation,” https://en.wikipedia.org/wiki/Z_notation, 2026.
- [55] The Dafny Project, “Dafny: A verification-aware programming language,” <https://dafny.org/>, 2026.
- [56] Cryspen, “hax: High-assurance translation of Rust to F* and Rocq,” <https://hax.cryspen.com/>, 2026.
- [57] K. Bhargavan, L. L. Hansen, F. Kiefer, J. Schneider-Bensch, and B. Spitters, “Formal security and functional verification of cryptographic protocol implementations in Rust,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS ’25)*. ACM, 2025, pp. 2729–2743.
- [58] B. Blanchet *et al.*, “ProVerif: Cryptographic protocol verifier in the formal model,” <https://bblanche.github.io/proverif/>, 2026.
- [59] Z. Li, J. Sun, L. Murphy, Q. Su, Z. Li, X. Zhang, K. Yang, and X. Si, “A survey on deep learning for theorem proving,” in *First Conference on Language Modeling (COLM 2024)*, 2024.
- [60] T. Hubert, R. S. Mehta, L. Sartran *et al.*, “Olympiad-level formal mathematical reasoning with reinforcement learning,” *Nature*, 2025.
- [61] B. Bayazit, Y. Li, and X. Si, “A case study on the effectiveness of LLMs in verification with proof assistants,” in *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages (LMPL 2025)*, 2025.
- [62] M. R. H. Misu, C. V. Lopes, I. Ma, and J. Noble, “Towards AI-assisted synthesis of verified Dafny methods,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 812–835, 2024.
- [63] J. Yao, Z. Zhou, W. Chen, and W. Cui, “Leveraging large language models for automated proof synthesis in Rust,” arXiv:2311.03739, 2023.
- [64] C. Yang, X. Li, M. R. H. Misu, J. Yao, W. Cui, Y. Gong, C. Hawblitzel, S. K. Lahiri, J. R. Lorch, S. Lu, F. Yang, Z. Zhou, and S. Lu, “AutoVerus: Automated proof generation for Rust code,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3454–3482, 2025.
- [65] E. Mugnier, E. Anaya Gonzalez, N. Polikarpova, R. Jhala, and Y. Zhou, “Laurel: Unblocking automated verification with large language models,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA1, pp. 1519–1545, 2025.
- [66] E. First, M. N. Rabe, T. Ringer, and Y. Brun, “Baldur: Whole-proof generation and repair with large language models,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023)*. ACM, 2023, pp. 1229–1241.
- [67] M. Lu, B. Delaware, and T. Zhang, “Proof automation with large language models,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)*. ACM, 2024, pp. 1509–1520.
- [68] N. Klaus, P. Tolmach, and J. Conejero, “A Rust-to-Lean verification pipeline with AI provers: An experience report,” arXiv:2605.30106, 2026.
- [69] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark, “DafnyBench: A benchmark for formal software verification,” arXiv:2406.08467, 2024.

- [70] J. Qin, A. Du, D. Zhang, M. Lentz, and D. Zhuo, “Can large language models verify system software? a case study using FSCQ as a benchmark,” in *Proceedings of the 20th Workshop on Hot Topics in Operating Systems (HotOS 2025)*. ACM, 2025.
- [71] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [72] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fix-points,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL ’77)*. ACM, 1977, pp. 238–252.
- [73] European Parliament and Council of the European Union, “Regulation (eu) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (cyber resilience act),” Official Journal of the European Union, L series, 2024.
- [74] ETSI, “EN 303 645: CYBER; cyber security for consumer Internet of Things — baseline requirements,” ETSI Standard, V3.1.3, 2024.
- [75] A. Armstrong, T. Bauereiss, B. Campbell, A. Reid, K. E. Gray, R. M. Norton, P. Mundkur, M. Wassell, J. French, C. Pulte, S. Flur, I. Stark, N. Krishnaswami, and P. Sewell, “ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–31, 2019.